

Brief de contexto - Paper PFSP

Contents

Brief de contexto — Paper PFSP (secciones que NO dependen de los resultados nuevos)

	1
1. Qué pide el enunciado del curso (sección 6, Trabajo de Investigación) . . .	1
2. El problema: PFSP — definición formal	2
3. Estado del arte — qué se sabe del PFSP y qué referencias usar	3
4. Marco teórico — Algoritmo Genético y Algoritmo Memético	5
5. Modelamiento y diseño — cómo se implementó cada pieza	6
6. Motivación del experimento con instancias “duras” (ta090, ta058) — para la Introducción y el diseño experimental	9
7. Resultados del ablation study (antes vs. después de las 4 mejoras) — dato ya disponible, útil como contexto aunque pertenece a la sección de Resultados	9
8. Cosas importantes que el equipo debe tener claras al redactar (para no cometer errores)	10
9. Referencias recopiladas hasta ahora (formato a ajustar a IEEE en la versión final)	11

Brief de contexto — Paper PFSP (secciones que NO dependen de los resultados nuevos)

Documento de trabajo interno para el equipo. Objetivo: que quien redacte las secciones de Introducción y Métodos del paper IEEE tenga todo el contexto necesario sin depender de las conversaciones donde se decidió cada cosa. Todo lo marcado como [VERIFICADO] proviene de una fuente concreta (código real, archivo real, o búsqueda web con URL); todo lo marcado como [PENDIENTE] es una decisión o dato que aún falta resolver con el equipo.

1. Qué pide el enunciado del curso (sección 6, Trabajo de Investigación)

Formato: artículo científico, plantilla IEEE Transactions. Extensión: 2000–3000 palabras. Debe ser 100% original. Copia de internet o de otra fuente = nota NCR.

Estructura mínima exigida: 1. Título, autor(es), afiliación institucional. 2. **Resumen (Abstract)** — problema, enfoque metodológico, resultados principales, relevancia del

estudio. **El profesor pidió explícitamente que el Abstract esté en inglés** (el resto del documento va en español). El resto de las secciones va con nombres en español salvo que la plantilla IEEE indique lo contrario. 3. **Introducción** — contexto del scheduling industrial, definición del PFSP, breve estado del arte, objetivos específicos. 4. **Métodos**: - Marco teórico: el problema PFSP y las dos metaheurísticas usadas (AG y Memético). - Modelamiento y diseño: representación, función de fitness, operadores genéticos elegidos y su justificación, operador de búsqueda local del Memético, mecanismo de reemplazo/elitismo y criterio de término. 5. **Resultados** — diseño experimental, ajuste y justificación de parámetros, tablas/gráficas comparando AG vs Memético, test estadístico. (Esta sección SÍ depende de los resultados nuevos que se están corriendo — no es parte de este brief.) 6. **Conclusiones** — principales aportes, ventajas y limitaciones de cada método, líneas de trabajo futuro. (También depende parcialmente de resultados nuevos — se puede dejar un esqueleto, no el contenido final.) 7. **Referencias** en formato IEEE.

Entrega: redactado en Overleaf o equivalente, avance semanal (30% de la nota), informe final (70% de la nota).

Este brief cubre en detalle las secciones **3 y 4** (Introducción y Métodos), que no dependen de los resultados de ta090/ta058 que se están corriendo en paralelo. Las secciones 5 y 6 se completan después, cuando esos resultados estén listos.

2. El problema: PFSP — definición formal

[VERIFICADO — enunciado del proyecto]

Hay n trabajos (jobs): $J_1, J_2, \dots, J_n$. Hay m máquinas: $M_1, M_2, \dots, M_m$. Todos los trabajos pasan por todas las máquinas en el mismo orden: primero M_1 , luego M_2 , ..., hasta M_m . Esto distingue al **flow shop** del **job shop** (donde cada trabajo puede tener su propio orden de máquinas — eso NO es lo que se resuelve en este proyecto).

Se conoce de antemano el tiempo de procesamiento $p(i,j)$ = tiempo que tarda el trabajo i en la máquina j .

Restricción clave que da nombre al problema ("Permutation"): el orden en que los trabajos entran a la máquina 1 debe ser el mismo orden en que entran a todas las demás máquinas. No se pueden "adelantar" ni "atrasar" trabajos entre estaciones — toda la instancia queda definida por una única permutación.

Objetivo: encontrar la permutación $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ que minimiza el makespan ($C_{\max}$): el tiempo en que termina el último trabajo en la última máquina.

Notación: J_i es el trabajo número i como entidad fija (trae sus propios tiempos $p(i,j)$, que nunca cambian). π es el orden buscado: π_k significa "el trabajo que ocupa la posición k en la secuencia". Ejemplo: si $\pi = (J_3, J_1, J_2)$, entonces $\pi_1=J_3$, $\pi_2=J_1$, $\pi_3=J_2$. El cromosoma del AG es exactamente π escrito como arreglo de índices de trabajos.

Cálculo del makespan — recurrencia y ejemplo numérico

[VERIFICADO — enunciado del proyecto]

Regla de cálculo: cada trabajo puede empezar en una máquina recién cuando (a) esa máquina quedó libre del trabajo anterior, Y (b) el propio trabajo ya terminó en la máquina previa.

$$C(i,j) = \max(C(i-1,j), C(i,j-1)) + p(i,j)$$

Ejemplo con 3 trabajos, 2 máquinas:

Trabajo	M1	M2
J1	5	3
J2	2	6
J3	4	4

Para $\pi = (J1, J2, J3)$:

Máquina 1: $C(J1,M1)=5$, $C(J2,M1)=\max(5,0)+2=7$, $C(J3,M1)=\max(7,0)+4=11$

Máquina 2: $C(J1,M2)=\max(0,5)+3=8$, $C(J2,M2)=\max(8,7)+6=14$, $C(J3,M2)=\max(14,11)+4=18$

Makespan de esta secuencia: $C_{\max} = 18$.

Este es exactamente el fitness que evalúa el AG y el Memético: dado un cromosoma (permutación), se recorre esta tabla y el makespan resultante es el valor a minimizar.

Por qué es un problema difícil

[VERIFICADO — enunciado del proyecto]

El espacio de búsqueda (todas las permutaciones posibles de n trabajos) crece como $n!$, lo que hace la fuerza bruta intratable para instancias medianas o grandes. Esto es lo que justifica el uso de metaheurísticas: no garantizan la solución óptima, pero exploran el espacio de forma inteligente y encuentran soluciones de buena calidad en tiempo razonable.

Dónde aparece este problema en la práctica

[VERIFICADO — enunciado del proyecto, ejemplo de motivación]

Manufactura (el ejemplo canónico: una fábrica textil donde cada pedido pasa, en el mismo orden, por corte → costura → planchado → empaquetado), líneas de ensamblaje, procesamiento de datos por lotes (pipelines), impresión, y programación de tareas en sistemas computacionales.

3. Estado del arte — qué se sabe del PFSP y qué referencias usar

Taillard (1993) — el benchmark estándar

[VERIFICADO — enunciado del proyecto, referencia base sugerida por el curso]

Eric Taillard publicó en 1993 el conjunto de instancias benchmark que hoy es el estándar de facto para comparar algoritmos de PFSP (y de otros problemas de scheduling). Son 120 instancias, agrupadas en 12 bloques de 10 instancias cada uno, con tamaños crecientes de n (trabajos) y m (máquinas): desde 20×5 hasta 500×20 .

Referencia (formato a verificar contra plantilla IEEE final): Taillard, E. (1993). “Benchmarks for basic scheduling problems.” *European Journal of Operational Research*.

Reeves (1995) — algoritmo genético aplicado a flowshop

[VERIFICADO — enunciado del proyecto, referencia base sugerida por el curso]

Reeves, C. R. (1995). “A genetic algorithm for flowshop sequencing.” *Computers & Operations Research*. Referencia clásica de aplicación de AG al flowshop, útil para justificar la elección de representación (permutación) y operadores.

Gmys (2022) — el estado del arte en métodos exactos

[VERIFICADO — arXiv 2012.09511, “Solving large permutation flow-shop scheduling problems on...” — mismo trabajo publicado como Gmys, J. (2022), “Exactly Solving Hard Permutation Flowshop Scheduling Problems on Peta-Scale GPU-Accelerated Supercomputers”, *INFORMS Journal on Computing*]

Este es el trabajo de referencia para métodos **exactos** (no metaheurísticos): un algoritmo de Branch-and-Bound paralelo (PBB@Cluster), ejecutado en clusters con cientos de GPUs, que logró resolver de forma exacta y demostrar la optimalidad de varias instancias de Taillard que llevaban décadas sin resolverse (las llamadas “23 instancias abiertas” de $m=20$ máquinas y $n \geq 50$ trabajos).

Cita textual relevante para justificar por qué las instancias que usa este proyecto (20×5 , 50×10 , 100×10) NO son parte de ese desafío: “instances defined by less than 20 machines are relatively easy to solve” — es decir, el propio paper de Gmys clasifica esas instancias como fáciles y ni siquiera reporta su tiempo de resolución.

Tiempos de resolución exacta reportados por Gmys para instancias de **20 máquinas** (las consideradas duras), con Branch-and-Bound paralelo en GPU:

Instancia	Tamaño	Tiempo (wall-clock)	GPUs usadas	CPU-tiempo equivalente (aprox.)
ta090	100×20	78 segundos	128	— (no reportado)
ta083	100×20	24 minutos	64	— (no reportado)
ta057	50×20	1h11	384	~8.5 años
ta052	50×20	1h54	384	~14 años
ta053	50×20	7h59	128	~19 años
ta058	50×20	13h17	256	~64 años
ta101	200×20	0h18	32	—

Instancia	Tamaño	Tiempo (wall-clock)	GPUs usadas	CPU-tiempo equivalente (aprox.)
ta107	200×20	0h06	32	—
ta108	200×20	28 segundos	32	—
ta109	200×20	100 segundos	32	—

ta058 fue la instancia más difícil del conjunto reportado: su demostración de optimalidad requirió explorar 339×10^{12} nodos del árbol de búsqueda.

Punto clave para la Introducción: el contraste entre este método exacto (que demuestra matemáticamente que no existe una solución mejor, pero requiere infraestructura de supercómputo) y las metaheurísticas (que no dan esa garantía, pero corren en hardware de consumo en segundos o minutos) es el eje motivacional de este proyecto.

[PENDIENTE] Formato de cita IEEE exacto para Gmys — falta confirmar si se cita como el paper de arXiv (2012.09511) o como la versión publicada en INFORMS Journal on Computing (2022); revisar cuál es la versión “oficial” citable según la norma del curso.

4. Marco teórico — Algoritmo Genético y Algoritmo Memético

[VERIFICADO — enunciado del proyecto + estructura real del código]

Algoritmo Genético (AG)

Metaheurística poblacional inspirada en la evolución biológica. Mantiene una población de soluciones candidatas (individuos), que evoluciona generación tras generación mediante: - **Selección:** elegir individuos con mejor fitness para reproducirse (más probabilidad de ser elegidos cuanto mejor es su Cmax). - **Cruce (crossover):** combinar dos individuos “padres” para generar un “hijo” que hereda características de ambos. - **Mutación:** alterar aleatoriamente un individuo para introducir diversidad y evitar estancamiento en óptimos locales. - **Reemplazo:** decidir qué individuos de la nueva generación reemplazan a la población anterior.

El AG es principalmente un mecanismo de **exploración:** recorre el espacio de soluciones de forma amplia, pero no refina localmente las soluciones que encuentra.

Algoritmo Memético

Extiende el AG agregando una etapa de **búsqueda local** (aplicada al mejor individuo, o a un subconjunto de los mejores, cada cierto número de generaciones). Esto agrega **explotación:** una vez que el AG encuentra una región prometedora del espacio de búsqueda, la búsqueda local refina la solución dentro de esa región hasta llegar a un óptimo local.

La comparación central de este proyecto es precisamente: **AG puro (solo exploración) vs. AG + búsqueda local (exploración + explotación)** — para cuantificar qué tanto aporta la etapa de refinamiento local.

5. Modelamiento y diseño — cómo se implementó cada pieza

[VERIFICADO — contra el código real del repositorio, `src/` y raíz del proyecto]

Representación del cromosoma

Un individuo es directamente la permutación π : un arreglo de n enteros, cada uno el índice de un trabajo, sin repetición. No hay una capa de codificación/decodificación adicional — el cromosoma ES la secuencia de producción.

Generación de números aleatorios (`src/aleatorios.py`)

Dos funciones base, envoltura de la librería `random` de Python: - `generar_real_aleatorio()`: número real en $[0, 1]$ — usado para decisiones probabilísticas (¿aplico cruce?, ¿aplico mutación?). - `generar_entero_aleatorio(minimo, maximo)`: entero en un rango dado, ambos extremos incluidos — usado para elegir posiciones, índices, puntos de corte.

Inicialización de la población (`src/poblacion.py`)

`generar_permutacion_aleatoria(n_trabajos)` implementa el **algoritmo de Fisher-Yates**: genera una permutación uniformemente aleatoria (sin sesgo hacia ningún orden particular) de los n trabajos. `inicializar_poblacion(tam_poblacion, n_trabajos)` genera la población inicial completa llamando a esta función `tam_poblacion` veces.

Mejora aplicada — heurística constructiva NEH: en vez de que toda la población inicial sea 100% aleatoria, se calcula una solución con la heurística NEH (Nawaz, Enscore y Ham, 1983) — determinista, ordena los trabajos por tiempo total de procesamiento descendente e inserta uno a uno en la posición que minimiza el makespan parcial — y se inserta como uno de los individuos de la población inicial (reemplazando a un individuo aleatorio). Esto da al AG/Memético un punto de partida de buena calidad desde la generación 0, en vez de partir de puro ruido.

Función de fitness (`src/fitness.py`)

`calcular_makespan(sequencia, tiempos)` construye la tabla completa $C(i,j)$ siguiendo exactamente la recurrencia mostrada en la sección 2 (no una versión optimizada en memoria que solo guarde la fila anterior) — decisión de diseño explícita para poder mostrar y defender la tabla completa frente al profesor, tal como aparece en el enunciado.

Verificación externa realizada: esta función fue verificada contra secuencias óptimas publicadas externamente (dataset de Olivier Brunner en Zenodo, DOI asociado al

registro 17028980), calculando el makespan de la secuencia óptima publicada y comparándolo contra el Cmax reportado — coincidencia exacta en las instancias verificadas. Esto confirma que la implementación del cálculo de makespan es correcta.

Selección (src/seleccion.py)

`seleccionar_por_torneo(poblacion, fitnesses, tam_torneo=2)`: selección por torneo binario — se eligen 2 individuos al azar de la población y se queda el de mejor fitness (menor Cmax). Es un método simple, con presión selectiva moderada y fácil de justificar frente al profesor.

Cruce (src/cruce.py)

`cruce_ox(padre1, padre2)`: **Order Crossover (OX)**. Copia un segmento contiguo de un padre y completa las posiciones restantes con el orden relativo de los trabajos del otro padre, saltando los ya copiados. Es el operador de cruce estándar para representaciones de permutación, porque garantiza que el hijo resultante sea una permutación válida (sin repetir ni omitir trabajos) — a diferencia de un cruce de un punto “clásico”, que rompería esa propiedad.

Mutación (src/mutacion.py)

`mutar_intercambio(individuo)`: **swap mutation** — intercambia la posición de dos trabajos elegidos al azar dentro de la secuencia. Operador simple que introduce diversidad sin alterar drásticamente la estructura de la solución.

Mejora aplicada — mutación adaptativa: la probabilidad de mutación no es fija durante toda la ejecución; decae linealmente desde el valor inicial configurado hasta un 10% de ese valor en la última generación. La intuición: al principio de la ejecución conviene más exploración (mayor diversidad, mayor probabilidad de mutación), y hacia el final, cuando la población ya convergió hacia buenas soluciones, conviene más explotación (menor probabilidad de mutación, para no destruir soluciones buenas ya encontradas).

Búsqueda local — el operador memético (src/busqueda_local.py)

`busqueda_local_insercion(individuo, tiempos)`: **insertion local search**, estrategia **first-improvement**. Para cada trabajo de la secuencia, se prueba moverlo (reinsertarlo) en cada una de las demás posiciones posibles; en cuanto se encuentra un movimiento que mejora el makespan, se aplica de inmediato y se reinicia la búsqueda desde ese nuevo punto. El proceso continúa hasta que ningún movimiento de reinserción produce mejora — es decir, hasta llegar a un óptimo local respecto a este vecindario de movimientos.

Mejora aplicada — búsqueda local sobre k mejores individuos: en vez de aplicar la búsqueda local solamente al mejor individuo de la población, se aplica a los k=2 mejores individuos cada vez que corresponde. Esto amplía la explotación sin concentrarla en un único punto del espacio de búsqueda.

Mejora aplicada — frecuencia de búsqueda local ajustada por tamaño de instancia: dado que cada pasada de búsqueda local es costosa computacionalmente (mientras más grande la instancia, más cara cada evaluación de makespan dentro de la búsqueda), la frecuencia con la que se aplica —cada cuántas generaciones— se ajusta según el tamaño de la instancia: se aplica más seguido en instancias chicas (donde es barata) y con menor frecuencia en instancias grandes (donde es cara), para mantener el tiempo total de cómputo manejable sin sacrificar la calidad de la búsqueda.

Nota honesta para el equipo: en una fase de exploración del proyecto se implementó también una variante **best-improvement** (evaluar todos los movimientos posibles de una pasada y aplicar el mejor, en vez de detenerse en el primero que mejora) y un operador de cruce alternativo **PMX** (Partially Mapped Crossover, Goldberg y Lingle, 1985). Ambos se probaron cualitativamente y se descartó su uso en la versión final: la mejora en RPD era marginal frente al aumento significativo en tiempo de cómputo. **No se guardaron números exactos de esa comparación** (fue una evaluación rápida, no un experimento formal) — si se menciona en el paper, debe presentarse como una decisión de diseño cualitativa, no citar cifras que no existen.

Reemplazo y elitismo (`src/reemplazo.py`)

`generar_nueva_poblacion(poblacion, fitnesses, tiempos, prob_cruza, prob_mutacion, con_elitismo=True)`: genera la nueva generación aplicando selección, cruce y mutación; con elitismo activado por defecto, lo que garantiza que el mejor individuo de la generación anterior sobreviva sin ser reemplazado por uno peor — evita que el algoritmo “pierda” accidentalmente la mejor solución encontrada hasta el momento.

Parámetros del programa

Ambos programas (`ag.py`, `memetico.py`) reciben como entrada: semilla aleatoria, instancia a resolver, tamaño de población, probabilidad de cruce, probabilidad de mutación, número de generaciones; el Memético además recibe la frecuencia de búsqueda local y (opcionalmente) `k_mejores`.

Métrica de comparación: RPD

$$RPD(\%) = ((C_{\text{max_obtenido}} - C_{\text{max_óptimo_o_mejor_conocido}}) / C_{\text{max_óptimo_o_mejor_conocido}}) \times 100$$

Nota conceptual importante para el paper: el upper bound (UB) / mejor conocido. Sí se puede alcanzar ($RPD=0\%$) — de hecho ya ocurrió en la instancia más chica del proyecto (ta001) en al menos una de las 30 corridas realizadas. Pero **superarlo** en instancias bien estudiadas de Taillard sería, en la práctica, señal de un error de cálculo, no un logro: esos UB provienen de décadas de investigación especializada, incluyendo métodos exactos que demuestran matemáticamente la optimalidad.

6. Motivación del experimento con instancias “duras” (ta090, ta058) — para la Introducción y el diseño experimental

[VERIFICADO — combinación del enunciado del proyecto + búsqueda de Gmys 2022]

Contexto para la Introducción (motivación general del trabajo): el estado del arte en métodos exactos para el PFSP (Branch-and-Bound paralelo, Gmys 2022) ha logrado resolver instancias que llevaban décadas abiertas, pero a costa de infraestructura de supercómputo — cientos de GPUs trabajando en paralelo durante horas. Esto ilustra el trade-off clásico entre métodos exactos (garantía de optimalidad, costo computacional que puede ser prohibitivo) y metaheurísticas (sin garantía de optimalidad, pero ejecutables con recursos de cómputo modestos, en tiempos mucho menores).

Contexto para el diseño experimental (por qué se agregaron ta090 y ta058 al conjunto de instancias): las tres instancias originales del proyecto (ta001, ta041, ta071; 5 y 10 máquinas) son consideradas “fáciles” incluso por el propio paper de Gmys, que ni reporta su tiempo de resolución exacta. Para poner a prueba el método frente a un desafío más representativo del estado del arte, se agregaron dos instancias de 20 máquinas, que sí forman parte del conjunto que Gmys reporta como difícil de resolver de forma exacta: - **ta090** (100 trabajos × 20 máquinas): la instancia más “accesible” del conjunto reportado por Gmys — resuelta en 78 segundos, pero usando 128 GPUs en paralelo. - **ta058** (50 trabajos × 20 máquinas): la instancia más costosa del conjunto reportado — resuelta en 13 horas y 17 minutos, usando 256 GPUs en paralelo, explorando 339×10^{12} nodos del árbol de búsqueda.

El objetivo es medir el RPD y el tiempo de ejecución del AG y el Memético en estas dos instancias, ejecutándose en un único núcleo de un procesador de escritorio (sin GPU, sin paralelismo), para argumentar que —aunque sin la garantía de optimalidad que ofrece el método exacto— una metaheurística ejecutada con recursos de cómputo accesibles puede seguir siendo una alternativa útil en escenarios donde no se requiere la garantía matemática de optimalidad, sino una solución suficientemente buena en un tiempo razonable.

[PENDIENTE] Los resultados concretos de RPD y tiempo para ta090/ta058 — experimento en curso al momento de escribir este brief, se agregan en una segunda entrega de este documento (o directamente en la sección de Resultados del paper).

7. Resultados del ablation study (antes vs. después de las 4 mejoras) — dato ya disponible, útil como contexto aunque pertenece a la sección de Resultados

[VERIFICADO — resultados/resumen.csv y resultados/antes_de_mejoras/resumen.csv, archivos reales del repositorio]

Este dato no es parte de Introducción/Métodos, pero se incluye acá porque ya está disponible y verificado, y da contexto de qué tan efectivas fueron las 4 mejoras (NEH, mutación adaptativa, k_mejores, frecuencia_bl por instancia) antes de sumar las instancias nuevas.

Antes de las 4 mejoras:

Instancia	Método	Cmax promedio	RPD promedio (%)	Tiempo promedio (s)
ta001	AG	1309.87	2.49	0.37
ta001	Memético	1295.97	1.41	0.44
ta041	AG	3375.50	12.86	1.65
ta041	Memético	3148.23	5.26	7.84
ta071	AG	6313.80	9.43	3.41
ta071	Memético	5867.47	1.69	65.44

Después de las 4 mejoras:

Instancia	Método	Cmax promedio	RPD promedio (%)	Tiempo promedio (s)
ta001	AG	1286.00	0.63	0.41
ta001	Memético	1284.97	0.55	0.85
ta041	AG	3134.03	4.78	2.23
ta041	Memético	3097.20	3.55	17.62
ta071	AG	5846.00	1.32	5.15
ta071	Memético	5806.87	0.64	78.60

Test de Wilcoxon pareado (RPD, AG vs Memético, misma semilla): diferencia estadísticamente significativa en las 3 instancias ($p \ll 0.05$ en todos los casos), favoreciendo siempre al Memético.

8. Cosas importantes que el equipo debe tener claras al redactar (para no cometer errores)

- **No confundir “mejor valor conocido” con “óptimo demostrado”:** en las instancias usadas en este proyecto (ta001, ta041, ta071, ta090, ta058), el campo `Optimal=1` en `best_known.csv` confirma que el UB es efectivamente el óptimo matemático demostrado ($LB=UB$), no solo una cota. Esto NO es cierto para todas las instancias de Taillard — varias (ej. ta051, ta081, ta111) siguen sin resolverse de forma exacta (`Optimal=0`), y ahí el “mejor conocido” es solo el mejor valor publicado hasta ahora, sin garantía de que sea el óptimo.
- **No inventar tiempos de B&B para instancias que no los tienen reportados:** se buscó explícitamente en la literatura (paper de Gmys y otros) y no existe un tiempo publicado y citable para ta001, ta041 ni ta071 — el paper de Gmys las considera “fáciles” y no las tabula. Solo hay tiempos reales para las instancias de 20 máquinas listadas en la sección 3.
- **Las mejoras 4 y 6 (best-improvement, PMX) no tienen datos numéricos guardados** — si se mencionan, debe quedar claro que fue una evaluación cualitativa exploratoria, no un experimento con cifras verificables.

- **El Abstract va en inglés** (pedido explícito del profesor); el resto del documento en español.
 - El código completo, con todas las funciones mencionadas en la sección 5, ya está implementado y probado — cada módulo de `src/` tiene su propio bloque de prueba (`if __name__ == "__main__":`).
-

9. Referencias recopiladas hasta ahora (formato a ajustar a IEEE en la versión final)

1. Taillard, E. (1993). "Benchmarks for basic scheduling problems." *European Journal of Operational Research*.
2. Reeves, C. R. (1995). "A genetic algorithm for flowshop sequencing." *Computers & Operations Research*.
3. Nawaz, M., Enscore, E. E., & Ham, I. (1983). "A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem." *Omega*. (Heurística NEH usada para inicializar la población.)
4. Goldberg, D. E., & Lingle, R. (1985). "Alleles, loci, and the traveling salesman problem." (Origen del operador de cruce PMX — mencionar solo si se decide incluir la mejora descartada en el paper.)
5. Gmys, J. (2022). "Exactly Solving Hard Permutation Flowshop Scheduling Problems on Peta-Scale GPU-Accelerated Supercomputers." *INFORMS Journal on Computing* (también disponible como preprint: arXiv:2012.09511).
6. Brunner, O. Dataset de secuencias óptimas para Taillard y VFR — Zenodo, DOI asociado al registro 17028980 (CC-BY-4.0). [PENDIENTE: confirmar formato de cita para un dataset, no un paper tradicional]

[PENDIENTE] Revisar si el curso exige un número mínimo de referencias, y si hay que agregar más papers de contexto (por ejemplo, algún survey de metaheurísticas para PFSP) — no confirmado en el enunciado revisado.